

Codex implementation plan for the current `audit.js`

This section converts the product spec into a step-by-step implementation path that Codex can follow safely in the current file.

Why this sequencing fits the current codebase

The current `audit.js` already has stable seams:

- `validateInput()` for intake
- `detectSignals()` for heuristics
- `runJsonModel()` for structured model calls
- `runSingleAssetAudit()` for single-pass auditing
- `sanitizeSingleAuditShape()` for deterministic cleanup
- `runCampaignFitAudit()` for cross-asset evaluation
- the final exported request handler for routing

That means the right move is an incremental refactor, not a full rewrite.

Non-negotiable implementation rules for Codex

Give Codex these rules before any patching begins:

1. Do not rewrite the file from scratch.
2. Preserve the existing endpoint contract unless the step explicitly changes it.
3. Keep all current behavior working until the replacement path is fully wired.
4. Add new functionality behind additive helpers first, then switch call sites.
5. Keep model output schema-locked and sanitize every model response before returning it.
6. Move final launch verdict logic into code, not the model.
7. Avoid UI-coupled backend field names unless intentionally public.
8. Keep campaign mode working throughout the migration.
9. Prefer pure helper functions over inline logic.
10. After each step, run lint/tests or at minimum perform a syntax check and review the diff.

Codex execution method

Use Codex in the two-step flow recommended by OpenAI:

- first Ask mode for implementation planning
- then Code mode for the patch

OpenAI's guidance says Codex works best when given structure, context, and room to iterate, and recommends starting larger changes in Ask mode before switching to Code mode. It also recommends prompts that read like GitHub issues, with file paths and concrete constraints. (openai.com)

Because Codex tasks run in isolated environments with reviewable diffs, treat each step below like a separate task or PR-sized branch. (openai.com)

Step 1 — Add richer intake without breaking the current API

Goal

Expand single-asset input context so later engines can score with traffic and market awareness, without breaking current callers.

What Codex should change

In `validateSingleInput(body)`, add optional fields with safe defaults:

- `channel`
- `traffic_source`
- `audience_temperature`
- `awareness_level`
- `market_category`
- `offer_model`
- `price_point`
- `sales_cycle`
- `claim_sensitivity`
- `competitive_maturity`
- `brand_proof_available`

Keep existing required fields:

- `copy`
- `copy_type`
- `goal`

Required behavior

- If fields are missing, preserve current behavior via defaults.
- Return a nested `context` object rather than scattering top-level fields.
- Do not require the frontend to send these yet.

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: extend `validateSingleInput()` to support an additive context object for future launch-readiness scoring.

Constraints:

- do not break the current endpoint contract
- preserve existing defaults for `copy_type` and `goal`
- add optional context fields with sane defaults

- return { mode, copy, copy_type, goal, context }
- keep campaign mode untouched in this step
- make the change minimal and readable

After planning, identify exactly which lines/functions you will touch and what follow-on changes are required so the file still runs.

Codex Code prompt

Implement the Step 1 plan in /mnt/data/audit.js.

Requirements:

- modify validateSingleInput() only as much as needed
- add a small helper if needed for normalized context defaults
- preserve backward compatibility
- do not yet change schemas or model prompts
- ensure exported handler still works with current callers

When done, summarize the diff and call out any future dependency created by the new context object.

Step 2 — Replace `detectSignals()` with a structured heuristic engine

Goal

Upgrade the current regex-based signal layer into a broader heuristic packet that later subsystems can use.

New signal groups to add

- `claims`
- `proof_markers`
- `authority_markers`
- `comparative_markers`
- `guarantee_markers`
- `urgency_markers`
- `ai_pattern_markers`
- `distinctness_markers`
- `offer_stack_markers`
- `risk_markers`

Specific detection targets

Add counts and sampled phrases for:

- superiority phrases (`best` , `better` , `only` , `unlike` , `more effective`)
- safety/trust phrases (`safe` , `trusted` , `confidence` , `certainty`)
- quantification (`%` , dollar values, counts, durations)
- testimonial indicators (`said` , `review` , `customer` , quoted testimonial blocks)
- external authority indicators (`doctor` , `study` , `certified` , `independent` , `expert`)
- guarantee language (`refund` , `money back` , `risk-free`)
- urgency framing (`now` , `today` , `limited` , `before` , `deadline`)
- AI-generic cadence markers (repetitive short-line contrast, overused marketing phrases, stacked rhetorical fragments)

Required behavior

- Preserve the current top-level `signals` object shape where possible.
- Add new nested sections instead of renaming existing ones.
- Keep it deterministic and cheap.

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: upgrade `detectSignals()` into a richer heuristic packet without introducing external dependencies.

Current goal:

- preserve existing sections (meta, cta, proof, offer, positioning, cadence)
- add new sections for claims, authority, comparisons, guarantees, urgency, AI-pattern risk markers, and distinctness markers
- use simple regex/string heuristics only
- keep runtime cheap and code readable

Please inspect `detectSignals()`, propose the exact new return shape, and explain how to add the heuristics without breaking existing consumers.

Codex Code prompt

Implement the `detectSignals()` upgrade in `/mnt/data/audit.js`.

Requirements:

- keep existing keys stable where possible
- add additive nested sections only
- avoid external packages
- create helper regex lists/constants if that improves readability
- do not modify model prompts yet
- preserve campaign behavior

When done, summarize the new signal groups and any assumptions.

Step 3 — Add an evidence classifier before model calls

Goal

Create a deterministic classifier that turns raw text + signals into a structured proof inventory.

New helper

Add `classifyEvidence(text, signals)` returning:

- `has_any_evidence`
- `evidence_types_present`
- `proof_density`
- `claim_to_proof_balance`
- `testimonial_strength`
- `authority_strength`
- `operational_proof_strength`
- `product_validation_strength`
- `mechanism_proof_strength`
- `substantiation_visibility`
- `evidence_gaps`

Required behavior

- Base this on heuristics only.
- Do not call the model here.
- Keep the output simple enough to pass into prompts.

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: add a new pure helper named `classifyEvidence(text, signals)`.

Purpose:

- create a deterministic evidence inventory before model scoring
- classify proof as testimonial, quantified, expert, mechanism, operational, product-validation, authority, or missing

Constraints:

- no external packages
- no model calls
- keep the logic heuristic and transparent
- shape the output so later prompts can consume it directly

Please propose the function signature, return schema, and where it should be called in the current file.

Codex Code prompt

Implement `classifyEvidence(text, signals)` in `/mnt/data/audit.js` and wire it into `buildSingleAuditPacket()`.

Requirements:

- add the new helper as a pure function
- include the evidence packet in the single-audit packet
- do not change the current JSON schema yet
- do not remove existing behavior

After patching, summarize how evidence classification is computed.

Step 4 — Split the single-pass audit into subsystem prompts

Goal

Keep `runJsonModel()` but stop using one giant single-asset prompt for all decisions.

New prompt constants to add

- `PERSUASION_AUDIT_SYSTEM_PROMPT`
- `PROOF_STRENGTH_SYSTEM_PROMPT`
- `SKEPTICISM_ENGINE_SYSTEM_PROMPT`
- `CLAIM_EXPOSURE_SYSTEM_PROMPT`
- `DECISION_SYNTHESIS_SYSTEM_PROMPT`

Strategy

Do not delete `SINGLE_AUDIT_SYSTEM_PROMPT` yet. Add the new prompts first.

First pass scope

For this step, only create the prompt constants and lightweight subsystem schemas. Do not switch the runtime wiring yet.

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: prepare the file for multi-engine auditing by adding subsystem prompt constants and matching JSON schemas.

Add these prompt constants:

- `PERSUASION_AUDIT_SYSTEM_PROMPT`
- `PROOF_STRENGTH_SYSTEM_PROMPT`
- `SKEPTICISM_ENGINE_SYSTEM_PROMPT`
- `CLAIM_EXPOSURE_SYSTEM_PROMPT`
- `DECISION_SYNTHESIS_SYSTEM_PROMPT`

Add matching lightweight schemas for each subsystem.

Constraints:

- do not delete the existing SINGLE_AUDIT_SYSTEM_PROMPT yet
- do not switch runtime behavior yet
- keep schema fields compact and additive
- optimize for later deterministic verdict resolution in code

Please plan the minimal schema fields for each subsystem based on the current product goals.

Codex Code prompt

Implement the subsystem prompt constants and subsystem schemas in /mnt/data/audit.js.

Requirements:

- leave current single-pass behavior untouched
- keep new schemas minimal but production-usable
- make field names stable and explicit
- keep prompts focused on JSON-only output

When done, summarize the new prompts and schema names.

Step 5 — Add subsystem sanitizers

Goal

Introduce deterministic sanitizers for each future model output before any runtime switch.

New helpers

- `sanitizePersuasionShape(result)`
- `sanitizeProofStrengthShape(result)`
- `sanitizeSkepticismShape(result)`
- `sanitizeClaimExposureShape(result)`
- `sanitizeDecisionSynthesisShape(result)`

Required behavior

- clamp all numeric fields
- sanitize enums
- trim string arrays
- ensure booleans are booleans
- provide safe fallbacks

Codex Ask prompt

You are editing /mnt/data/audit.js.

Task: add sanitizer helpers for the new subsystem outputs.

Constraints:

- follow the style of `sanitizeSingleAuditShape()` and `sanitizeCampaignFitShape()`
- use `clampInt()` and `sanitizeEnum()` where appropriate
- keep outputs safe for deterministic code-based verdict resolution later
- do not switch any runtime calls yet

Please identify the minimal sanitization rules needed for each subsystem.

Codex Code prompt

Implement subsystem sanitizer helpers in `/mnt/data/audit.js`.

Requirements:

- do not alter current endpoint behavior
- keep the sanitizer style consistent with the current file
- ensure all new fields have sensible fallbacks

At the end, summarize what each sanitizer protects against.

Step 6 — Add subsystem runners while preserving current output

Goal

Create new runner functions without yet changing the public response shape.

New helpers

- `runPersuasionAudit(packet)`
- `runProofStrengthAudit(packet)`
- `runSkepticismAudit(packet)`
- `runClaimExposureAudit(packet)`
- `runDecisionSynthesis(packet, subsystemResults)`

Required behavior

- all call `runJsonModel()`
- all pass compact JSON packets
- all sanitize their results immediately
- no endpoint output changes yet

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: add runtime helpers for each new subsystem without changing the current public API yet.

Constraints:

- each runner should call `runJsonModel()`
- each runner should immediately sanitize the returned JSON
- current single-pass audit path must continue to work untouched in this step
- design these helpers so Step 7 can switch `runSingleAssetAudit()` to the new flow

Please map the exact packet shape each runner should receive from the current single-asset packet.

Codex Code prompt

Implement the new subsystem runner helpers in `/mnt/data/audit.js`.

Requirements:

- keep current behavior unchanged
- add only the new helpers and any small packet-builder helpers they need
- avoid duplicate packet construction where possible

When done, summarize the new runner functions and their inputs.

Step 7 — Replace `runSingleAssetAudit()` with the new multi-engine flow

Goal

Switch single-asset auditing from one model pass to a staged tribunal flow.

New internal flow

1. validate input
2. build single audit packet
3. run persuasion audit
4. run proof strength audit
5. run skepticism audit
6. run claim exposure audit
7. synthesize
8. resolve final verdict in code
9. return a richer audit object

Important

At this step, the response shape can change, but preserve legacy fields where practical so the UI does not explode.

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: replace the internals of `runSingleAssetAudit()` so it uses the new multi-engine sequence instead of the old single-pass prompt.

Constraints:

- preserve the current function name
- keep the old response fields available if reasonably possible for UI compatibility
- include subsystem outputs in the returned audit object in a way that future UI can use
- do not remove campaign mode
- do not yet remove the old prompt constant unless no longer needed

Please describe the compatibility strategy before patching.

Codex Code prompt

Implement the Step 7 refactor in /mnt/data/audit.js.

Requirements:

- run the new subsystem pipeline inside runSingleAssetAudit()
- preserve legacy summary fields where possible
- include additive fields for persuasion_scores, proof_strength_index, skepticism_engine, claim_exposure, and decision_basis
- keep code readable and modular

When done, summarize the compatibility choices and any fields the frontend may need to adopt later.

Step 8 — Add deterministic verdict resolver logic in code

Goal

Ensure the final launch verdict is code-resolved, not model-resolved.

New helper

Add `resolveLaunchVerdict(context, subsystemResults)`.

Verdict ladder

- `No-Go`
- `Hold`
- `Conditional Go`
- `Test-Only Go`
- `Go`
- `Scale-Ready`

Minimum gating rules

- If proof overall < 6 and audience is cold, max verdict = `Conditional Go`
- If product validation < 5 and superiority positioning is present, max verdict = `Hold`
- If claim risk is elevated and substantiation visibility is low, max verdict = `Hold`
- If offer score < 6, max verdict = `Test-Only Go`
- If AI pattern risk is high and competitive maturity is high, `safe_to_scale = false`

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: add a deterministic verdict resolver for the new subsystem outputs.

Requirements:

- final verdict must be computed in code, not trusted from model output
- use a ladder: No-Go, Hold, Conditional Go, Test-Only Go, Go, Scale-Ready
- implement the minimum gates provided in the spec
- keep the rules readable and easy to extend

Please propose a clean rule-ordering strategy so later rules do not become brittle.

Codex Code prompt

Implement `resolveLaunchVerdict(context, subsystemResults)` in `/mnt/data/audit.js` and wire it into the single-asset audit flow.

Requirements:

- final verdict logic must be deterministic code
- return verdict, verdict_confidence, safe_to_test, safe_to_scale, primary_blocker, and highest_risk_failure_mode
- keep naming stable and explicit

When done, summarize the gating rules in the diff summary.

Step 9 — Add a legacy compatibility adapter for the current UI

Goal

Your frontend likely expects fields like `dimension_scores`, `total_score`, `reason`, and `fix_instruction`. Create an adapter so the new engine can still feed the current UI while you phase in the richer panels.

New helper

Add `toLegacySingleAuditShape(newAudit)`.

Required behavior

- map persuasion outputs into legacy `dimension_scores`
- compute a legacy-compatible `total_score`
- derive `weakest_dimension` , `reason` , and `fix_instruction`
- preserve `big_idea` if still present in persuasion results

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: create a compatibility adapter so the new multi-engine audit can still satisfy the current UI contract.

Constraints:

- do not delete the richer internal output
- generate the old shape from the new audit object
- keep the mapping deterministic and readable

Please describe which legacy fields can be preserved exactly, which need derivation, and which may need temporary placeholders.

Codex Code prompt

Implement `toLegacySingleAuditShape(newAudit)` in `/mnt/data/audit.js` and use it so the endpoint remains compatible with the current UI.

Requirements:

- preserve current frontend stability as much as possible
- keep the richer audit object available alongside the legacy adapter if feasible
- document mapping assumptions in code comments only where helpful

When done, summarize the mapping strategy.

Step 10 — Expand campaign mode to use the richer single-asset results

Goal

Keep campaign mode alive after the single-asset engine changes.

What to change

Update `buildCampaignSummaryPacket()` so it consumes the richer audit output, not only the legacy fields.

Add campaign summary inputs like:

- `launch_verdict`
- `copy_strength_score`
- `launch_safety_score`
- `primary_blocker`
- `highest_risk_failure_mode`
- `proof_strength_index.overall`
- `skepticism_engine.ai_pattern_risk`

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: upgrade campaign packet building so campaign-fit can use the richer single-asset audit results.

Constraints:

- preserve current campaign mode behavior
- keep the packet compact
- prefer summaries over raw nested payloads
- do not redesign the campaign schema yet beyond what is needed

Please identify the smallest useful set of new cross-asset fields to include.

Codex Code prompt

Implement the campaign summary packet upgrade in `/mnt/data/audit.js`.

Requirements:

- preserve current campaign auditing flow
- add richer asset summary fields that support future continuity diagnostics
- keep payload size reasonable

When done, summarize the added campaign summary fields.

Step 11 — Remove or retire the old single-pass path

Goal

Only after the new flow is stable, retire unused single-pass pieces.

What Codex should remove only if unused

- `SINGLE_AUDIT_SYSTEM_PROMPT`
- `SINGLE_AUDIT_SCHEMA`
- any dead helper logic related only to the old path

Required behavior

- do not remove legacy UI compatibility helpers
- do not remove campaign mode

Codex Ask prompt

You are editing `/mnt/data/audit.js`.

Task: identify dead code from the old single-pass audit path now that the new multi-engine flow is active.

Constraints:

- remove only code that is truly unused
- do not break current endpoint output
- preserve compatibility helpers and campaign mode

Please list what can be safely removed and why before patching.

Codex Code prompt

Remove only dead code from the retired single-pass path in `/mnt/data/audit.js`.

Requirements:

- keep endpoint behavior stable
- do not remove anything still referenced
- prefer small, reviewable cleanup

When done, summarize what was deleted and confirm no remaining references.

Recommended order of Codex runs

Do not hand Codex all 11 steps at once. Use separate tasks or branches in this order:

1. Step 1–3 together
2. Step 4–6 together
3. Step 7 alone
4. Step 8 alone
5. Step 9–10 together
6. Step 11 cleanup

This keeps each change in the size range OpenAI recommends for Codex to handle reliably. (openai.com)

Review checklist after every Codex patch

Ask Codex or yourself to verify:

- endpoint still returns valid JSON
 - all new model calls use schemas
 - all new model outputs are sanitized
 - campaign mode still works
 - no existing field names broke accidentally
 - verdict logic is code-driven
 - no unused schema or prompt constants remain after cleanup
 - diff is narrow and readable
-

Recommended next build output

The next practical artifact should be a **Codex task pack** containing:

- one Ask prompt and one Code prompt for each implementation step
- explicit acceptance criteria per step
- expected diff surfaces per step
- a migration map from legacy output to richer decision output

That pack can then be used directly in Codex task-by-task.

Yes — and that's the right way to do it.

I mapped your current `audit.js` into a **Codex-ready step-by-step refactor plan** built around the file's actual seams: `validateInput()`, `detectSignals()`, `runJsonModel()`, `runSingleAssetAudit()`, the sanitizers, and the campaign helpers. The plan is incremental, so Codex can patch the existing file instead of doing a risky rewrite. That matches OpenAI's guidance to give Codex structured, well-scoped tasks, start larger work in Ask mode before Code mode, and frame prompts like GitHub issues with concrete file paths and constraints.

The implementation sequence is now in the canvas document: **"DEC 2026 Pre-Traffic Decision Engine Build Spec."** I added a dedicated **Codex implementation plan** with:

- 11 ordered refactor steps
- a separate Ask prompt and Code prompt for each step
- exact functions to add or modify
- compatibility rules so the current endpoint and UI do not break early
- a deterministic verdict resolver phase so launch decisions move into code rather than staying model-only

The sequence starts by expanding intake context and upgrading `detectSignals()`, then adds a deterministic `classifyEvidence()` layer, then introduces subsystem prompts and sanitizers, then swaps `runSingleAssetAudit()` to a multi-engine flow, and only after that retires the old single-pass path. That staged approach is important because Codex tasks run in isolated environments with reviewable diffs, so smaller PR-sized patches are more reliable than one giant migration.

The highest-leverage execution order is:

- . Step 1–3 together
- !. Step 4–6 together
- }. Step 7 alone
- }. Step 8 alone
- i. Step 9–10 together
- i. Step 11 cleanup

That keeps each Codex task narrow enough to review properly while preserving your current app behavior.

The best next move is to start with **Step 1–3 as the first Codex task batch**:

- add `context` to `validateSingleInput()`
- upgrade `detectSignals()`
- add `classifyEvidence():contentReference[oaicite:2]{index=2}cket()`

That's the cleanest foundation for everything else.